魔方理念

> 版本：2026-04-15
> 适用范围：NewLife.Cube 第三代（主力）、兼容第二代存量系统

---

1. 我们为什么重塑魔方

魔方的定位始终不是“做一个页面框架”，而是做一个**快速开发平台**：

- 让团队把时间花在“业务差异化”上，而不是重复搭后台基础页面
- 用统一的平台能力覆盖绝大多数通用管理场景
- 在技术快速演进的前端生态中，保持后端能力稳定、可复用、可扩展

一句话定义：

魔方 = 面向系统管理后台的“默认可用 + 按需增强”的生产力底座。

---

2. 三代演进与不变核心

2.1 第一代（2010 起）：ASP.NET WebForm

- 技术特征：服务端视图渲染
- 价值：快速形成后台开发方法论
- 现状：已停止使用

2.2 第二代：ASP.NET Core MVC

- 技术特征：MVC 模式，前后端耦合相对降低
- 价值：承接第一代经验，增强可维护性
- 现状：进入弱化阶段，后续不再新增功能（以存量兼容为主）

2.3 第三代（当前主力）：ASP.NET Core 前后端分离

- 技术特征：统一 WebAPI + 多前端皮肤/技术栈复用
- 价值：后端一次开发，多端共享
- 现状：主战场，持续演进

2.4 三代变化中的“不变”

无论实现技术如何更替，魔方坚持同一条产品理念：

1. **默认生成**：系统后台绝大多数页面可由平台直接生成默认版式与能力
2. **按需定制**：只有少量复杂业务页才需要深度定制
3. **渐进增强**：在默认可用基础上，做局部优化与功能增强，而非推翻重写

> 实践判断：后台场景中，绝大多数是标准 CRUD（增删改查）+ 查询筛选 + 导入导出 + 权限控制。

3. 重塑后的产品定位

面向未来的魔方，不是绑定某个前端框架，而是构建“**协议稳定、实现多样**”的平台能力层。

3.1 平台定位

- **后端**：提供统一、稳定、语义清晰的 API 能力
- **前端**：允许百花齐放（Vue/React/Arco/TDesign/NaiveUI/Metronic…）
- **终端**：不仅是 Web，还可对接桌面客户端（如 WinForms/WPF/Avalonia）

3.2 架构主张

- 一套业务接口，服务多种展示形态
- 接口优先于页面，能力优先于实现
- 保持核心行为一致，允许 UI 风格差异

---

4. 魔方的核心理念（必须长期坚持）

4.1 “80/20”效率原则

- 用平台能力覆盖 80%~90% 的常规后台需求
- 为 10%~20% 的复杂场景预留清晰扩展点

4.2 “先通用、后个性”

- 先让系统完整跑起来（登录、权限、菜单、实体 CRUD、导入导出）
- 再对关键业务页面做定向增强

4.3 “一致性优先”

- 统一响应结构：`ApiResponse<T>`
- 统一认证入口：`/Auth/*`
- 统一站点信息：`/Cube/SiteInfo`
- 统一实体能力：列表、详情、字段元数据、导入导出、上传、图表

4.4 “配置驱动能力开关”

通过配置启用/关闭能力，减少分叉代码：

- `AllowLogin`（密码登录）
- `AllowRegister`（注册入口）
- `EnableSms` / `EnableMail`（验证码登录与找回通道）
- `AllowPlainPassword`（是否允许明文密码）
- `LoginLogo` / `LoginBackground` / `LoginTip` / `Copyright` / `Registration`

4.5 “安全默认可用”

- 支持 Challenge-Response（`/Auth/Challenge` + RSA-OAEP/SHA-256）
- `challengeId` 限时、一次性使用，防重放
- 保留兼容降级路径，但推荐安全优先

---

5. 登录域能力：重塑中的第一优先

登录能力是平台入口，也是“可配置产品化能力”的典型样板。目标不是“堆功能”，而是“可组合、可切换、可落地”。

5.1 必备能力面

- 用户名密码登录
- 手机验证码登录
- 邮箱验证码登录
- 用户注册（含验证码注册）
- 忘记密码（验证码重置）
- 第三方 OAuth 登录
- OAuth 回跳后绑定注册

5.2 配置化策略

- 各能力可按项目场景开关控制
- 前端根据 `/Auth/LoginConfig` 动态渲染入口
- 通过 `providers` 动态渲染第三方登录按钮

5.3 多皮肤一致体验

- 不同皮肤可有不同视觉与交互细节
- 但流程与语义保持一致：同一接口、同一状态码、同一业务结果

---

6. 工作台首页与权限菜单体系：重塑中的第二大方向

工作台首页不是“一个页面”，而是用户进入系统后的**能力入口编排层**。
它承接“菜单、权限、角色、功能动作”四类核心对象，是魔方权限体系的外化界面。

6.1 菜单模型：菜单不仅是导航，也是权限载体

- 一个系统通常按业务域拆分为二级、三级菜单
- 菜单树中包含三类节点：
  - **目录**：用于组织结构，可能不可直接访问
  - **菜单页**：可见、可进入的业务页面
  - **功能点**：如查看、新增、编辑、删除等动作，通常不直接展示为导航菜单

> 这意味着：有些节点“在菜单管理里存在，但在用户导航里不可见”，它们仍然参与权限判定。

6.2 Membership 权限体系：用户—角色—菜单功能

魔方采用 Membership 风格的授权模型，本质是基于角色的权限控制（RBAC）并细化到菜单功能级别：

- **用户可绑定多个角色**（一人多角）
- 角色中可区分**主要角色**与附加角色
- 每个角色在每个菜单（或功能点）上可配置动作权限，例如：
    - 查看
    - 新增
    - 编辑
    - 删除

可把角色理解为：

角色 = 一组菜单与功能动作权限的集合。

系统授权时，管理员只需要给用户分配一个或多个角色，即可完成权限下发。

6.3 工作台首页的设计目标

- **按权可见**：用户只看到自己有权访问的菜单与功能入口
- **按角聚合**：多个角色权限合并后生成最终可用能力
- **按需呈现**：目录、菜单页、功能点分层展示，避免首页信息噪声
- **按域扩展**：新增业务模块时，通过新增菜单节点与角色授权即可接入

6.4 与“快速开发平台”定位的关系

工作台首页 + 菜单权限体系，是“默认生成 + 局部增强”理念在管理后台中的直接落地：

- 平台默认生成菜单结构与权限配置骨架
- 项目团队只需在关键业务点做局部定制
- 通过角色授权快速完成不同岗位用户的差异化工作台

---

7. 列表页与表单页体系：重塑中的第三大方向

如果说登录解决“谁能进来”，工作台解决“看到什么入口”，那么列表页体系解决的就是“如何高效完成日常业务操作”。

7.1 一一对应关系：表 → 实体 → 控制器 → 页面

魔方的后台页面构建遵循稳定映射关系：

- 后台数据表对应实体类
- 实体类对应控制器
- 控制器对应前端页面
- 页面由**列表页 + 表单页**组成，二者一一对应

这套映射关系让平台可以自动生成默认可用的管理页面，显著降低后台开发门槛。

7.2 列表页的默认能力基线

列表页默认以表格承载数据展示，核心能力包括：

- **分页展示**：默认每页 20 行，可按皮肤与项目需求调整
- **排序能力**：多数基础字段支持排序（扩展字段可按需关闭）
- **搜索能力**：支持关键字搜索 + 多条件搜索
- **行级操作**：典型为编辑、删除、查看详情
- **批量操作**：支持多选后执行批量启用、批量停用、批量刷新等

可理解为：

列表页 = 数据可视化 + 检索过滤 + 批量执行 + 权限约束。

7.3 列表页结构：工具栏、表格主体、分页区

- **上方工具栏**：搜索框、添加按钮、业务操作按钮、批量操作入口
- **中间表格主体**：核心信息承载区，支持排序、列渲染、状态呈现
- **下方分页区**：与后端分页接口协同，支持强分页能力

其中，工具栏按钮与行内操作按钮都应由权限系统动态控制显隐与可用状态。

7.4 权限驱动的交互策略

列表页上的操作不是“固定显示”，而是“按权限呈现”：

- 有新增权限才显示“添加”
- 无删除权限时，删除按钮可隐藏或置灰
- 无编辑权限时，可降级为“详情查看”而非编辑入口

这与 Membership 角色授权体系保持一致：页面行为由角色权限组合结果决定。

7.5 图表与列表的协同方式

历史 MVC 版本常在列表页上方/下方嵌入图表（以 ECharts 为主，曲线图最常见，柱状图/饼图次之）。

第三代前后端分离场景中，推荐采用“表格与图表分区或多标签切换”策略：

- 避免图表与表格在同一区域拥挤
- 保持数据浏览与分析视图切换自然
- 让不同皮肤按自身设计系统灵活实现

7.6 表格增强能力（平台加分项）

为提升复杂业务场景表达力，表格需要支持可扩展增强：

- **行级条件着色**：按状态整行标红/黄/绿等
- **列级条件着色**：按列值规则应用颜色和样式

- **模板渲染列**：后端可输出 HTML 模板片段，前端按约定替换数据后安全渲染

这些能力可让后端工程师在不深入前端细节的情况下，完成更强的业务表达。

7.7 用户自定义列（差异化增强，非必需）

这是列表页体验的“加分项”，不是硬性必选项：

- 支持用户勾选列显隐
- 支持调整列顺序
- 支持保存个性化配置到浏览器本地

该能力可显著提升不同岗位用户的使用效率与满意度。

---

8. 搜索区体系：重塑中的第四大方向

搜索区是列表页“可用”与“好用”的分水岭。它决定了用户能否在最短路径内找到目标数据。

8.1 关键字搜索约定：`Q` 是固定协议

魔方约定关键字搜索统一使用 `Q` 参数回传后端接口，这是长期稳定的协议约束。

- 前端将用户输入统一放入 `Q`
- 后端按约定对字符串字段执行模糊匹配
- 返回匹配结果用于列表展示

这条约定让不同皮肤在搜索行为上保持一致，避免“同平台不同规则”。

8.2 时间区间搜索：`dtStart` + `dtEnd`

除关键字外，时间区间是后台场景的第二高频条件。

- 搜索条件一般包含开始时间与结束时间
- 用于筛选某一时间段内的业务数据
- 若未特别指定字段，后端默认按“更新时间”进行区间过滤

这一默认策略兼顾通用性与可解释性，降低配置复杂度。

8.3 下拉筛选：单选、多选与数据源联动

搜索区常见还有下拉筛选，通常分为单选与多选两类：

- **单选下拉**：建议提供“全部”选项；不选择等同于“全部”（即不加过滤）
- **多选下拉**：必须具备“清空”能力，便于快速回到全量视图
- **远程数据源下拉**：当候选项来自其他数据表时，后端提供专用接口返回数据源

目标是让筛选表达能力更强，同时操作成本更低。

8.4 搜索区与页面元数据驱动

魔方列表页本质是“字段元数据驱动的动态渲染”。搜索区同样遵循这一原则：

- 搜索控件类型来自字段定义（文本、枚举、远程字典、时间等）
- 列表页与表单页可按字段配置差异化显示
- 页面功能按钮与区块显隐可由配置统一控制

这让平台具备“少写前端代码、快速拼装页面”的核心优势。

8.5 元数据接口演进：从多次 `GetFields` 到单接口聚合

当前 API 版本下，列表页通常需要多次调用后端 `GetFields` 获取不同模式元数据。
后续建议演进为**单接口聚合返回**：

- 一次返回整个页面设置（Page Setting）
- 同时返回列表页/表单页在不同模式下的字段显示配置
- 前端可缓存该元数据，减少重复请求与初始化耗时

这种方式更适合前后端分离架构下的高性能与高一致性要求。

8.6 前端可控页面设置（Page Setting）

将页面设置能力前置到前端配置层，可覆盖更多真实业务诉求，例如：

- 某些场景隐藏导航栏
- 某些场景隐藏工具栏
- 按业务状态切换字段可见性与布局

这属于魔方过去十多年沉淀下来的高频能力，应继续保留并标准化。

---

9. 表单页体系：重塑中的第五大方向

表单页是数据写入的唯一入口，承接"新增 / 编辑 / 详情"三种模式，也是后台页面里与用户交互最密集的环节。

9.1 三种使用场景：新增、编辑、详情

表单页在实际系统中通常复用同一个页面，按进入时的上下文切换模式：

- **新增模式**：点击列表页工具栏的"添加"按钮进入，提交后调用新增接口向数据库插入一行
- **编辑模式**：点击列表行的"编辑"进入，回填已有数据，提交后调用更新接口
- **详情模式**：与编辑页复用同一布局，仅隐藏提交按钮；只读展示，不做额外专属页面

> **设计取向**：新增与编辑共用模板，详情复用编辑模板，能复用就不分叉——这与"默认生成、少写代码"的平台理念完全一致。

9.2 前端呈现形式：弹窗或路由跳转

在前后端分离架构下，表单页的呈现形式由前端自行决定：

- 简单数据表可以弹窗（Dialog/Drawer）形式快速填写与关闭
- 字段较多或业务较复杂时，跳转到独立路由页面更合适
- 两种方式均应支持，皮肤可按设计系统偏好选择

9.3 基础字段控件：枚举与关联数据

表单页字段控件由元数据驱动，常见场景包括：

- **枚举单选**：与状态枚举关联，直接下拉选择
- **关联他表单选**：下拉 + 模糊搜索，按关键字过滤候选项后选择
- **关联他表多选**：支持多选，是过去做得不够好的方向，前端皮肤应重点提升这一体验
- **文件上传**：部分字段需要附件，表单页应内置文件上传支持

9.4 文件上传设计原则

文件上传遵循"独立先传模式"（参见核心接口架构）：

- 用户选择文件后立即上传，拿到 `attId`
- 主表单提交时携带 `attId` 完成绑定
- 后端保存主记录后自动关联附件

这种方式避免表单提交与文件上传耦合，减少超时、重传等问题。

9.5 主子表数据设计约定

魔方不推荐"主子表同页一次新增"的方式，原因是字段增多后交互复杂度急剧上升。
历史验证有效的简化方案是：

1. 先完成新增主表数据的表单
2. 保存后进入编辑模式，再通过子表区块操作子表数据

未来期望：前端皮肤如果能更好地支持"主表保存后的子表联动编辑区块"，体验将大幅提升，鼓励各皮肤在此做差异化增强。

9.6 表单页能力总结

可以把表单页的平台责任理解为：

表单页 = 字段元数据渲染 + 关联数据联动 + 文件上传 + 权限约束 + 三态复用（新增/编辑/详情）。

平台提供默认可用的全功能表单骨架，业务层在特殊字段与复杂交互处做局部增强。

---

10. 高级操作区：导入导出与数据管理

列表页工具栏的"高级"下拉菜单，承载了一批低频但关键的数据管理能力。
这些功能面向运维与开发场景为主，**导出用得最多，其余属于便利性选项，非必选实现。**

10.1 导出：最高频的高级操作

- 支持将当前查询条件下的数据导出为 **Excel**（`.xlsx` / CSV 兼容）或 **CSV**
- 导出遵循当前搜索条件与排序，不是整表导出
- 前端通过 `window.open(url)` 直接触发下载，不占用 AJAX 通道
- 数据量超过阈值（如 10 万行）时后端自动启用分片导出

- 最大导出行数受 `CubeSetting.MaxExport` 控制

10.2 导入：自动识别多种格式

- 支持从浏览器上传 Excel、CSV 及压缩包（后端自动识别格式）
- 后端将文件内容匹配到实体字段，依次写入数据库
- 字段名与列头匹配策略：优先精确匹配，再模糊匹配显示名

10.3 批量删除与条件清空

这是高级菜单里的"危险操作"区，提供两种删除粒度：

- **选中删除**：删除列表左侧多选的行
- **条件删除（全部）**：删除当前搜索条件下所有满足条件的数据
- **安全约束**：条件删除要求至少携带一个过滤条件，裸删全表的请求会被后端拒绝，防止误操作

10.4 清空数据表

- 清空整张数据表，属于高危不可逆操作
- 仅对具有相应权限的用户可见，且建议前端强制二次确认

10.5 备份与恢复

- **备份**：将整张表的数据导出为文件，下载到浏览器本地
- **恢复**：从浏览器上传备份文件，恢复到数据库中

这两项功能为开发与运维提供轻量级数据迁移/快照能力，属于便利功能，非系统级 HA 方案。

10.6 能力优先级说明

功能	使用频率	是否必选
导出 Excel / CSV	高	是
导入 Excel / CSV	中	推荐
批量删除（选中）	低	推荐
条件删除（全部）	很低	可选
清空数据表	极低	可选
备份与恢复	极低	可选

---

11. 代码生成与零代码启动：魔方的核心生产力理念

魔方最根本的价值命题：

只要有数据表和实体类，就可以得到对应的默认管理页面，不需要任何额外开发。
只有在需要定制时，才按定制需求的大小做不同程度的开发。

11.1 最短路径：数据模型 → 代码生成 → 启动即可用

魔方推荐的标准开发流程：

1. 建立 Model 数据模型文件（一张表对应一段配置）
2. 通过代码生成器生成实体类与控制器
3. 编译并重启项目
4. 魔方自动扫描新控制器，生成对应菜单
5. 打开菜单，即可看到数据表格，并直接完成增删改查

这意味着：**哪怕数据库有 100 张表，导出模型文件、人工核对、代码生成、一次编译重启之后，就能得到 100 个可用的后台页面**，每个页面均具备完整的查看、新增、编辑、删除能力。

11.2 快速原型：边沟通需求边生成页面

这套能力极大地提升了原型阶段的沟通效率：

- 与用户沟通需求的同时，同步建立数据模型
- 一轮需求沟通结束后，立即可以生成页面给用户查看效果
- 页面默认带有完整增删改查功能，足够支撑需求验证
- 发现调整点后，只需针对局部页面做对应修改

11.3 定制成本极低：大多数需求改三四行代码

绝大多数业务定制需求，在魔方体系下成本极低：

定制类型	典型操作
隐藏某个字段	在列表页或表单页配置中标记不显示
字段颜色区分	添加行/列条件着色配置，2~3 行代码
输出图表	后端准备 ECharts 配置结构返回，前端直接渲染
字段校验增强	在控制器或实体类中添加少量验证逻辑

这些操作通常只需要修改两三行、三四行代码，不需要重新搭页面。

11.4 深度定制：为关键页面投入精力

每个项目通常有三到五个核心页面，需要精细化定制，魔方同样支持：

- 前端工程师接管，做独立页面满足高密度交互需求
- 引入数据大屏模板，为用户呈现可视化分析页面
- 通过独立路由和自定义组件，与平台默认页面共存

大多数页面走"零代码默认生成"，少量核心页面走"精细化定制"——这是魔方最理想的开发节奏。

11.5 插件化：皮肤与功能的可扩展机制

魔方支持插件化能力，插件分为两类：

- **皮肤插件**：替换前端 UI 风格，适配不同用户口味与品牌要求，各皮肤彼此独立可切换
- **功能插件**：安装后为系统增加新的业务能力模块，不影响核心平台稳定性

插件化机制保证了：平台核心简洁，扩展能力丰富，升级互不干扰。

---

12. 用户分层理念：人人都能用魔方，各取所需

魔方的终极目标不是"只服务高端开发者"，而是**让不同技术背景的人都能在魔方体系下高效工作**。

根据技术能力，魔方服务三类用户，每类用户都有清晰的价值交付路径。

12.1 入门用户：只需会改 Model 文件

这类用户不需要了解后端开发，也不需要了解前端开发。
只需依葫芦画瓢，修改 `model.xml` 数据模型文件：

- 增加/删减字段
- 设定字段类型、长度、显示名称
- 配置枚举值或关联关系

保存文件，代码生成器一跑，编译重启，就能得到一套完整的后台管理系统。

这类用户的魔方价值：零编码成本，快速拥有属于自己的数据管理系统。

12.2 后端工程师：借助魔方聚焦业务逻辑

这类用户不擅长前端开发，但熟悉后端。
魔方让他们可以**通过后端代码控制前端的呈现与定制**：

- 修改控制器定制列表页展示逻辑，如字段颜色、排序、搜索条件
- 准备 ECharts 配置数据，由前端自动渲染图表
- 定制新增/编辑接口，加入业务校验与处理逻辑
- 无需了解 Vue/React，只写 C# 就能完成全部后台管理页面的定制

魔方的前端界面，对这类用户而言只是一个"数据可视化与维护工具"，
他们真正的精力在于业务逻辑开发、数据分析与处理。

这类用户的魔方价值：不用写前端，也能做出满足业务需要的完整管理系统。

12.3 全栈工程师：前后端通吃，聚焦用户体验

这类用户兼具后端与前端能力，可以做更精细化的定制：

- 后端：数据模型、实体类、控制器接口定制，以及复杂业务逻辑
- 前端：接管关键页面，独立开发高交互体验的定制页、数据大屏
- 在魔方默认页面覆盖 80% 常规需求的基础上，专注那 20% 的高价值精细页面

这类用户的魔方价值：快速交付常规页面，把稀缺的全栈能力投入到真正需要差异化的地方。

12.4 三层分工的核心价值

```
入门用户      —>  model.xml  —>  代码生成  —>  立即可用
后端工程师    —>  后端定制代码（C#）  —>  前端自动呈现
全栈工程师    —>  全局定制  —>  精细页面 + 数据大屏
```

魔方的用户分层理念与"默认生成 + 按需增强"完全一致：
平台能力兜底，人的能力加分，每一层都有清晰的投入产出路径。

---

13. 第三代的关键价值主张

13.1 对研发团队

- 后端研发只需聚焦一套 API
- 前端研发可自由选择技术栈与 UI 风格
- 降低跨项目重复建设成本

13.2 对业务交付

- 项目启动更快：默认后台能力开箱可用
- 需求变更更稳：通过配置/局部增强响应变化
- 多终端延展更自然：Web 与桌面共享业务能力

13.3 对平台演进

- 能力可以模块化沉淀（认证、页面逻辑、字段映射、工具库）
- 可持续兼容存量接口，降低升级摩擦

---

14. 重塑边界与取舍

为了保持平台清晰度，重塑需要明确“做什么、不做什么”。

14.1 我们要做

- 持续强化第三代前后端分离能力
- 统一并稳定核心接口语义
- 强化配置驱动与默认页面生成能力
- 保证主流登录/注册/找回/OAuth 场景可直接落地

14.2 我们暂不做

- 不回到第一代技术路线
- 不为第二代新增重大能力（仅必要维护与兼容）

- 不为单一前端框架做平台层绑定

---

15. 未来设计准则（用于评估新需求）

任何新增能力，建议按以下问题做评审：

1. 是否能服务“多数项目”的共性需求？
2. 是否能通过配置开关控制，而非强耦合写死？
3. 是否保持接口语义稳定并兼容现有调用方？
4. 是否可被多前端/多终端复用？
5. 是否遵循“默认生成 + 局部增强”的平台哲学？

若以上大部分答案为“是”，应纳入平台能力；否则优先以项目级定制实现。

---

16. 一句话共识（建议对外统一表达）

魔方是一个以“快速交付后台系统”为目标、以“统一接口驱动多端呈现”为路径、以“默认生成 + 按需增强”为方法的开发平台。

它既服务今天的交付效率，也服务明天的技术多样性。

---

17. 参考文档

- `Doc/Api/核心接口架构.md`
- `Doc/Api/登录页接口与功能设计.md`